

E.A.6.6 - Edge Coloring

1 Modellazione

Introduciamo le variabili

$$x_{e,i} \quad e \in E, i \in \{1, 2, 3\}$$

$x_{e,i} = \top \iff$ l'arco e è colorato con il colore i .

Sia $\mathcal{T}$ l'insieme di tutti i cicli di lunghezza 3 presenti nel grafo G . Un elemento di $\mathcal{T}$ è un set di tre archi $\{e_1, e_2, e_3\}$ che connettono tre vertici distinti (assumiamo no self-loops) $u, v, w \in V$:

$$\mathcal{T} = \{\{e_1, e_2, e_3\} \subseteq E \mid e_1 = \{u, v\}, e_2 = \{v, w\}, e_3 = \{w, u\} \text{ con } u \neq v \neq w\}$$

La formula $\Phi(G)$ è composta dalla congiunzione di:

1. ogni arco deve avere almeno un colore

$$\bigwedge_{e \in E} (x_{e,1} \vee x_{e,2} \vee x_{e,3})$$

2. per ogni triangolo e per ogni colore, tre archi di un triangolo non possono essere dello stesso colore

$$\bigwedge_{\{e_1, e_2, e_3\} \in \mathcal{T}} \bigwedge_{i=1}^3 (\neg x_{e_1,i} \vee \neg x_{e_2,i} \vee \neg x_{e_3,i})$$

Come per Schur's Lemma, non è necessario imporre che ogni arco abbia esattamente un colore. Se si assegnano più colori a un arco senza violare i vincoli sui triangoli, la soluzione è comunque valida (è sufficiente scegliere uno qualsiasi dei colori assegnati).

2 Istanziamento

$$E = \{\{G1, H\}, \{G1, E\}, \{H, E\}, \{H, I\}, \{H, A\}, \{I, A\}, \{I, B\}, \{A, E\}, \{A, S\}, \\ \{E, D\}, \{S, D\}, \{S, B\}, \{S, C\}, \{B, J\}, \{B, C\}, \{B, G2\}, \{J, G2\}, \{D, C\}, \{C, G2\}\}$$

L'insieme dei triangoli $\mathcal{T}$ presenti nel grafo è composto dai seguenti 7 cicli di lunghezza 3:

$$\begin{aligned}\mathcal{T} = \{ & \{G1, H\}, \{G1, E\}, \{H, E\}, \\ & \{H, I\}, \{H, A\}, \{I, A\}, \\ & \{H, E\}, \{H, A\}, \{A, E\}, \\ & \{B, J\}, \{J, G2\}, \{B, G2\}, \\ & \{B, C\}, \{C, G2\}, \{B, G2\}, \\ & \{S, D\}, \{D, C\}, \{S, C\}, \\ & \{S, B\}, \{B, C\}, \{S, C\} \}\end{aligned}$$

La formula proposizionale $\Phi(G_{1,1})$ in CNF è la congiunzione dei seguenti vincoli:

vincoli di colorazione per archi

- $(x_{\{G1,H\},1} \vee x_{\{G1,H\},2} \vee x_{\{G1,H\},3})$
- $(x_{\{G1,E\},1} \vee x_{\{G1,E\},2} \vee x_{\{G1,E\},3})$
- $(x_{\{H,E\},1} \vee x_{\{H,E\},2} \vee x_{\{H,E\},3})$
- $(x_{\{H,I\},1} \vee x_{\{H,I\},2} \vee x_{\{H,I\},3})$
- $(x_{\{H,A\},1} \vee x_{\{H,A\},2} \vee x_{\{H,A\},3})$
- $(x_{\{I,A\},1} \vee x_{\{I,A\},2} \vee x_{\{I,A\},3})$
- $(x_{\{I,B\},1} \vee x_{\{I,B\},2} \vee x_{\{I,B\},3})$
- $(x_{\{A,E\},1} \vee x_{\{A,E\},2} \vee x_{\{A,E\},3})$
- $(x_{\{A,S\},1} \vee x_{\{A,S\},2} \vee x_{\{A,S\},3})$
- $(x_{\{E,D\},1} \vee x_{\{E,D\},2} \vee x_{\{E,D\},3})$
- $(x_{\{S,D\},1} \vee x_{\{S,D\},2} \vee x_{\{S,D\},3})$
- $(x_{\{S,B\},1} \vee x_{\{S,B\},2} \vee x_{\{S,B\},3})$
- $(x_{\{S,C\},1} \vee x_{\{S,C\},2} \vee x_{\{S,C\},3})$
- $(x_{\{B,J\},1} \vee x_{\{B,J\},2} \vee x_{\{B,J\},3})$
- $(x_{\{B,C\},1} \vee x_{\{B,C\},2} \vee x_{\{B,C\},3})$
- $(x_{\{B,G2\},1} \vee x_{\{B,G2\},2} \vee x_{\{B,G2\},3})$
- $(x_{\{J,G2\},1} \vee x_{\{J,G2\},2} \vee x_{\{J,G2\},3})$
- $(x_{\{D,C\},1} \vee x_{\{D,C\},2} \vee x_{\{D,C\},3})$
- $(x_{\{C,G2\},1} \vee x_{\{C,G2\},2} \vee x_{\{C,G2\},3})$

vincoli sui triangoli

- **triangolo** $\{\{G1, H\}, \{G1, E\}, \{H, E\}\}$:
 - $(\neg x_{\{G1, H\}, 1} \vee \neg x_{\{G1, E\}, 1} \vee \neg x_{\{H, E\}, 1})$
 - $(\neg x_{\{G1, H\}, 2} \vee \neg x_{\{G1, E\}, 2} \vee \neg x_{\{H, E\}, 2})$
 - $(\neg x_{\{G1, H\}, 3} \vee \neg x_{\{G1, E\}, 3} \vee \neg x_{\{H, E\}, 3})$
- **triangolo** $\{\{H, I\}, \{H, A\}, \{I, A\}\}$:
 - $(\neg x_{\{H, I\}, 1} \vee \neg x_{\{H, A\}, 1} \vee \neg x_{\{I, A\}, 1})$
 - $(\neg x_{\{H, I\}, 2} \vee \neg x_{\{H, A\}, 2} \vee \neg x_{\{I, A\}, 2})$
 - $(\neg x_{\{H, I\}, 3} \vee \neg x_{\{H, A\}, 3} \vee \neg x_{\{I, A\}, 3})$
- **triangolo** $\{\{H, E\}, \{H, A\}, \{A, E\}\}$:
 - $(\neg x_{\{H, E\}, 1} \vee \neg x_{\{H, A\}, 1} \vee \neg x_{\{A, E\}, 1})$
 - $(\neg x_{\{H, E\}, 2} \vee \neg x_{\{H, A\}, 2} \vee \neg x_{\{A, E\}, 2})$
 - $(\neg x_{\{H, E\}, 3} \vee \neg x_{\{H, A\}, 3} \vee \neg x_{\{A, E\}, 3})$
- **triangolo** $\{\{B, J\}, \{J, G2\}, \{B, G2\}\}$:
 - $(\neg x_{\{B, J\}, 1} \vee \neg x_{\{J, G2\}, 1} \vee \neg x_{\{B, G2\}, 1})$
 - $(\neg x_{\{B, J\}, 2} \vee \neg x_{\{J, G2\}, 2} \vee \neg x_{\{B, G2\}, 2})$
 - $(\neg x_{\{B, J\}, 3} \vee \neg x_{\{J, G2\}, 3} \vee \neg x_{\{B, G2\}, 3})$
- **triangolo** $\{\{B, C\}, \{C, G2\}, \{B, G2\}\}$:
 - $(\neg x_{\{B, C\}, 1} \vee \neg x_{\{C, G2\}, 1} \vee \neg x_{\{B, G2\}, 1})$
 - $(\neg x_{\{B, C\}, 2} \vee \neg x_{\{C, G2\}, 2} \vee \neg x_{\{B, G2\}, 2})$
 - $(\neg x_{\{B, C\}, 3} \vee \neg x_{\{C, G2\}, 3} \vee \neg x_{\{B, G2\}, 3})$
- **triangolo** $\{\{S, D\}, \{D, C\}, \{S, C\}\}$:
 - $(\neg x_{\{S, D\}, 1} \vee \neg x_{\{D, C\}, 1} \vee \neg x_{\{S, C\}, 1})$
 - $(\neg x_{\{S, D\}, 2} \vee \neg x_{\{D, C\}, 2} \vee \neg x_{\{S, C\}, 2})$
 - $(\neg x_{\{S, D\}, 3} \vee \neg x_{\{D, C\}, 3} \vee \neg x_{\{S, C\}, 3})$
- **triangolo** $\{\{S, B\}, \{B, C\}, \{S, C\}\}$:
 - $(\neg x_{\{S, B\}, 1} \vee \neg x_{\{B, C\}, 1} \vee \neg x_{\{S, C\}, 1})$
 - $(\neg x_{\{S, B\}, 2} \vee \neg x_{\{B, C\}, 2} \vee \neg x_{\{S, C\}, 2})$
 - $(\neg x_{\{S, B\}, 3} \vee \neg x_{\{B, C\}, 3} \vee \neg x_{\{S, C\}, 3})$

3 Codec

```

1  #include "Encoder.hpp"
2  #include "Problem.hpp"
3  #include <string>
4  #include <vector>
5
6  class EdgeColoring : public Problem {
7  private:
8      struct Triangle {
9          int e1, e2, e3;
10     };
11
12     int numVertices;
13     int numEdges;
14     std::vector<std::pair<int, int>> edges;
15     std::vector<Triangle> triangles;
16
17     inline std::string vName(int e, int i) const {
18         return "x_" + std::to_string(e) + "_" + std::
19             to_string(i);
20     }
21 public:
22     // il costruttore riceve i nodi e la lista degli archi, e
23     precalcola i triangoli
24     EdgeColoring(int V, const std::vector<std::pair<int, int>>& E)
25         : numVertices(V), numEdges(E.size()), edges(E) {
26
27         // matrice di adiacenza che mappa (u, v) all'indice
28         dell'arco
29         std::vector<std::vector<int>> adj(V, std::vector<int>
30             (V, -1));
31         for (int i = 0; i < numEdges; ++i) {
32             int u = edges[i].first;
33             int v = edges[i].second;
34             adj[u][v] = i;
35             adj[v][u] = i; // grafo non orientato
36         }
37
38         // troviamo tutti i triangolo
39         for (int i = 0; i < V; ++i) {
40             for (int j = i + 1; j < V; ++j) {
41                 if (adj[i][j] != -1) {
42                     for (int k = j + 1; k < V; ++k) {
43                         if (adj[i][k] != -1 && adj[j][k] !=
44                             -1) {
45                             triangles.push_back({adj[i][j],
46                                 adj[j][k], adj[i][k]});
47                         }
48                     }
49                 }
50             }
51         }
52     }
53 }

```

```

49     void generateConstraints(Encoder& encoder,
50                             EncodingContext& ctx,
51                             int tId, int tCount) override {
52         // ogni arco deve avere almeno un colore (1, 2 o 3)
53         for (int e = tId; e < numEdges; e += tCount) {
54             std::vector<int> edgeLits;
55             for (int i = 1; i <= 3; ++i) {
56                 edgeLits.push_back(encoder.getVar(vName(e, i)
57                                     ));
58             }
59             encoder.writeDirectClause(edgeLits, ctx);
60         }
61         // per ogni triangolo e per ogni colore,
62         // tre archi non possono essere dello stesso colore
63         // Partizionato per indice del triangolo
64         for (int t = tId; t < triangles.size(); t += tCount)
65         {
66             int e1 = triangles[t].e1;
67             int e2 = triangles[t].e2;
68             int e3 = triangles[t].e3;
69             for (int i = 1; i <= 3; ++i) {
70                 int lit1 = -encoder.getVar(vName(e1, i));
71                 int lit2 = -encoder.getVar(vName(e2, i));
72                 int lit3 = -encoder.getVar(vName(e3, i));
73             }
74             encoder.writeDirectClause({lit1, lit2, lit3},
75                                     ctx);
76         }
77     }
78 };

```